

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Федерального государственного автономного образовательного учреждения
высшего образования**

**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ МИСИС»**

Институт компьютерных наук

Кафедра Инженерной кибернетики

Лабораторная работа №1

по дисциплине «Современные интеллектуальные сетевые сервисы»

Проектирование AI-сервиса на FastAPI

Выполнил:

Студент группы МПИ 25-1-2

Д.А. Комарова

Проверил:

Преподаватель

М.Е. Корнет

Оценка: _____

Москва, 2026

Содержание

1	Цель работы	2
2	Задачи работы	2
3	Исходные данные	2
4	Требуемые endpoints	3
5	Теоретическая часть	3
5.1	Что такое REST API	3
5.2	Место API в интеллектуальной системе	4
5.3	Синхронные и асинхронные операции	4
5.4	Формат данных: JSON	4
5.5	HTTP-методы	4
5.6	Структура HTTP-запроса	4
5.7	Path- и query-параметры	5
5.8	Валидация данных и Pydantic	5
5.9	HTTP-коды ответов	5
5.10	Обработка ошибок	6
5.11	OpenAPI и Swagger UI	6
5.12	Логирование и наблюдаемость	6
5.13	Архитектурные риски	6
6	Создание проекта	7
7	Создание виртуального окружения	7
8	Создание FastAPI-приложения	7
9	Первый endpoint — /health	8
10	Swagger UI	8
11	OpenAPI	8
12	Задание 1. Добавление информации о модели	9
13	Проектирование входных данных	9
14	Pydantic	9
15	Реализация модели	10
16	Постпроцессинг	11
17	Задание 2. Реализуйте POST /predict	11
18	Проверка через Swagger	11
19	Модель ответа (Response Model)	12

20 Задание 3. Организовать хранение результатов	13
21 Path Parameters	13
22 Задание 4. Получение прогноза	13
23 Query Parameters	14
24 Задание 5. Фильтрация	15
25 Задание 6. Валидация Query Parameters	15
26 Задание 7. Проверка бизнес ограничений	15
27 Задание 8. Возвращаемые статусы	16
28 OpenAPI как контракт	16
29 Задание 9. Добавьте описание endpoints	16
30 Задание 10. Тестирование API	17
31 Задание 11. Проанализируйте API как контракт	17
32 Задание 12. Разделение приложения на модули	17
33 Требования к отчёту	18
34 Критерии оценки	19
35 Скриншоты работы приложения	19

1 Цель работы

Цель практической работы — научиться проектировать программную архитектуру AI-сервиса, использующего модель машинного обучения, определить место API в общей ИИ-системе и реализовать REST API на FastAPI, изучив основные принципы программного взаимодействия между AI-моделью и внешними информационными системами.

2 Задачи работы

В ходе работы необходимо:

- определить потребителей AI-сервиса;
- определить формат входных и выходных данных;
- спроектировать контур инференса;
- разделить синхронные и асинхронные операции;
- определить основные программные компоненты системы;
- выбрать протоколы взаимодействия;
- сформировать структурированный контракт REST API для AI-сервиса;
- определить способы хранения результатов;
- определить требования к логированию и наблюдаемости;
- выявить основные архитектурные риски;
- создать приложение на FastAPI и реализовать REST endpoints;
- изучить методы GET и POST, передачу данных через JSON, использование path- и query-параметров;
- описывать входные и выходные схемы с помощью Pydantic и выполнять валидацию входных данных;
- возвращать корректные HTTP-коды и обрабатывать ошибки;
- изучить автоматически создаваемую OpenAPI-документацию и протестировать API через Swagger UI;
- реализовать минимальный прототип API на FastAPI.

3 Исходные данные

Банк разрабатывает сервис агроскоринга. Система должна оценивать риск при работе с сельскохозяйственными предприятиями. Пользователь или другая информационная система передаёт сведения о хозяйстве: регион, площадь посевов, тип основной культуры, погодные показатели, историю платежей, количество просроченных платежей, размер текущей задолженности.

На основании этих данных модель машинного обучения должна определить риск профиля предприятия. Результат должен содержать числовую оценку риска, категорию риска и рекомендацию оператору банка.

Пример ответа:

```
1 {  
2     "risk_score": 0.73,  
3     "risk_level": "high",  
4     "recommendation": "Additional check required"  
5 }
```

Листинг 1: Пример ответа /predict

Сервис должен позволять:

- проверить, работает ли API;
- получить информацию о модели;
- выполнить оценку риска одного хозяйства;
- получить сохранённый результат прогноза;
- получить несколько последних прогнозов;
- корректно сообщать клиенту об ошибках.

Предполагается, что в будущем этим API будут пользоваться: CRM банка, веб-портал, мобильное приложение, внутренние банковские сервисы, BI-система.

4 Требуемые endpoints

API должен иметь следующие endpoints:

Таблица 1: Список endpoints API

Метод	Endpoint	Назначение
GET	/health	Проверка работоспособности API
GET	/model-info	Информация о модели
POST	/predict	Оценка риска хозяйства
GET	/predictions/{request_id}	Получение прогноза по ID
GET	/predictions	Список прогнозов

5 Теоретическая часть

5.1 Что такое REST API

REST API — способ взаимодействия программных систем через HTTP. Клиент отправляет запрос (HTTP Request), сервер обрабатывает его и возвращает ответ (HTTP Response).

Клиенту не нужно знать: на каком языке написана модель, где она находится, является ли модель scikit-learn или PyTorch, сколько функций выполняется внутри сервера. Клиенту достаточно знать контракт API.

5.2 Место API в интеллектуальной системе

AI-сервис редко существует изолированно. Как правило, он встроен в более крупную информационную систему. Типовая схема:

Пользователь / UI → Внешняя система (CRM, ERP, мобильное приложение)
→ HTTP / JSON → API-слой (FastAPI)
→ Контур инференса: предобработка → модель → постобработка
→ Хранилище результатов: БД, лог, очередь

API в этой схеме выполняет три роли:

1. **Точка входа** — единый интерфейс для всех внешних потребителей.
2. **Изолятор** — скрывает детали реализации модели (фреймворк, версию, способ загрузки).
3. **Контракт** — фиксирует форматы входных и выходных данных, коды ответов и правила обработки ошибок.

Благодаря этому модель можно заменить, переобучить или перенести на другой сервер, не меняя код клиентов.

5.3 Синхронные и асинхронные операции

В AI-сервисе операции делятся на два класса.

Синхронные — клиент ждёт ответ в рамках одного запроса. Подходят для быстрых моделей (например, инференс за 50–500 мс).

Асинхронные — клиент получает идентификатор задачи и забирает результат позже. Нужны для тяжёлых моделей, пакетной обработки или длительных вычислений.

Выбор между ними — архитектурное решение, влияющее на таймауты, нагрузку и требования к хранилищу.

5.4 Формат данных: JSON

Обмен данными между клиентом и AI-сервисом выполняется в формате JSON. Это текстовый формат, который легко читается человеком, поддерживается всеми языками программирования и удобен для различных структур данных (объекты, массивы, числа, строки, булевы значения).

5.5 HTTP-методы

В этой работе используются два основных метода. **GET** используется преимущественно для получения информации. **POST** используется для передачи данных серверу и создания результата.

5.6 Структура HTTP-запроса

Упрощённо HTTP-запрос можно представить следующим образом:

```
1 METHOD + URL
2 HEADERS
3 BODY
```

Листинг 2: Структура HTTP-запроса

Например:

```
1 POST /predict
2 Content-Type: application/json
3
4 {
5     "farm_id": "FARM-100",
6     "area_ha": 2500
7 }
```

Листинг 3: Пример POST-запроса

5.7 Path- и query-параметры

Данные можно передавать не только в теле запроса, но и прямо в URL.

Path-параметры — часть пути, идентифицируют конкретный ресурс, например: GET /predictions/123.

Query-параметры — идут после знака ?, обычно используются для фильтрации, сортировки, пагинации: GET /predictions?limit=10&offset=0®ion=Krasnodar.

Различие важно для проектирования контракта: path-параметр описывает *что* запрашивается, query-параметр уточняет *как* это подаётся (сколько, в каком порядке, с какими фильтрами).

5.8 Валидация данных и Pydantic

Клиент может прислать некорректные данные: пропустить обязательное поле, передать строку вместо числа, указать отрицательную площадь.

Чтобы сервис не падал и не выполнял инференс на ошибочных данных, входные и выходные структуры описываются явно — с помощью Pydantic-моделей:

```
1 from pydantic import BaseModel, Field
2
3 class PredictRequest(BaseModel):
4     farm_id: str
5     region: str
6     crop_type: str
7     area_ha: float = Field(gt=0)
8
9 class PredictResponse(BaseModel):
10     risk_score: float
11     risk_level: str
12     model_version: str
```

Листинг 4: Pydantic-модели запроса и ответа

Pydantic автоматически проверяет типы и ограничения и возвращает понятную ошибку при несоответствии. Это одновременно и защита сервиса, и документация контракта.

5.9 HTTP-коды ответов

Сервер сообщает о результате не только телом ответа, но и HTTP-кодом. Основные группы:

Таблица 2: Основные HTTP-коды

Код	Значение	Пример в AI-сервисе
200	OK	успешный инференс
201	Created	создана асинхронная задача
400	Bad Request	некорректное тело запроса
404	Not Found	задача с таким id не найдена
422	Unprocessable Entity	ошибка валидации Pydantic
500	Internal Server Error	сбой модели или кода
503	Service Unavailable	модель ещё не загружена

Корректные коды — часть контракта: клиент по ним понимает, нужно ли повторять запрос, исправлять данные или сообщать об ошибке пользователю.

5.10 Обработка ошибок

Ошибки бывают трёх типов. Ошибки клиента — некорректный JSON, отсутствует поле, неверный тип; возвращается 400 или 422 с описанием проблемы. Ошибки бизнес-логики — например, запрошен прогноз для несуществующего хозяйства; возвращается 404 или 409. Внутренние ошибки — падение модели, недоступность БД; возвращается 500, а детали пишутся в лог, но не отдаются клиенту.

5.11 OpenAPI и Swagger UI

FastAPI автоматически строит OpenAPI-спецификацию — машиночитаемое описание всех endpoints, схем запроса и ответа, кодов ошибок. На её основе генерируется Swagger UI — интерактивная страница, доступная по адресу `/docs`, где можно увидеть все endpoints, посмотреть схемы Pydantic, отправить тестовый запрос прямо из браузера и получить ответ и код. Это одновременно и документация для разработчиков, и инструмент тестирования, и основа для генерации клиентов на других языках.

5.12 Логирование и наблюдаемость

Для того чтобы обеспечить воспроизводимость, отследить деградацию модели и проверить, какая версия модели дала конкретный ответ, фиксируют входящие запросы (метод, путь, идентификатор клиента, время), результат инференса (версия модели, ключевые признаки, предсказание), ошибки (тип, стек, входные данные без персональных данных) и метрики (задержка, доля ошибок, количество запросов).

5.13 Архитектурные риски

Даже простой AI-сервис сталкивается с типовыми рисками:

Таблица 3: Архитектурные риски и меры

Риск	Проявление	Мера
Долгий инференс	таймауты у клиента	асинхронная обработка, очередь
Некорректные данные	падение модели	валидация Pydantic
Смена версии модели	непредсказуемые ответы	версионирование API и модели
Перегрузка	рост задержек	лимиты, кэширование, горизонтальное масштабирование
Отсутствие логов	невозможность разбора инцидента	структурированное логирование
Утечка данных	нарушение приватности	минимизация логируемых полей

Их выявление на этапе проектирования дешевле, чем устранение в проде.

6 Создание проекта

Создайте каталог `agro_api/`. Базовая структура:

```

1 agro_api/
2   main.py
3   schemas.py
4   model.py
5   requirements.txt

```

Листинг 5: Базовая структура проекта

7 Создание виртуального окружения

Рекомендуется создать отдельное окружение Python:

```
1 python -m venv venv
```

Активация:

```
1 venv\Scripts\activate
```

Установите зависимости:

```
1 pip install fastapi uvicorn
```

8 Создание FastAPI-приложения

Создайте файл `main.py` и добавьте:

```

1 from fastapi import FastAPI
2
3 app = FastAPI(

```

```
4     title="Agro Scoring API",
5     description="API for agricultural risk scoring",
6     version="1.0.0"
7 )
```

Листинг 6: Инициализация FastAPI-приложения

Здесь создаётся объект `app = FastAPI()`, который представляет наше API-приложение.

9 Первый endpoint — /health

Добавьте:

```
1 @app.get("/health")
2 def health():
3     return {
4         "status": "ok"
5     }
```

Листинг 7: Endpoint /health

Запустите приложение:

```
1 uvicorn main:app --reload
```

Откройте `http://127.0.0.1:8000/health`. Ожидаемый результат:

```
1 {"status": "ok"}
```

10 Swagger UI

Откройте `http://127.0.0.1:8000/docs`. FastAPI автоматически создаёт интерактивную документацию API. В ней должен появиться GET `/health`. Нажмите **Try it out**, затем **Execute**. Обратите внимание на Request URL, Response body, Response code и Response headers.

11 OpenAPI

FastAPI автоматически формирует описание API по стандарту OpenAPI. Откройте `http://127.0.0.1:8000/openapi.json` — вы увидите JSON-документ, описывающий ваш API. Упрощённо:

```
1 {
2     "openapi": "3.x",
3     "info": {
4         "title": "Agro Scoring API"
5     },
6     "paths": {
7         "/health": {}
8     }
9 }
```

Листинг 8: Фрагмент OpenAPI-спецификации

12 Задание 1. Добавление информации о модели

Создайте endpoint GET `/model-info`. Он должен возвращать:

```
1 {  
2     "model_name": "agro-risk-model",  
3     "model_version": "1.0",  
4     "model_type": "risk-scoring",  
5     "status": "ready"  
6 }
```

Листинг 9: Ответ `/model-info`

Пример реализации:

```
1 @app.get("/model-info")  
2 def model_info():  
3     return {  
4         "model_name": "agro-risk-model",  
5         "model_version": "1.0",  
6         "model_type": "risk-scoring",  
7         "status": "ready"  
8     }
```

Листинг 10: Endpoint `/model-info`

Ответьте в отчёте: чем `/health` отличается от `/model-info`?

Ответ: `/health` проверяет только работоспособность сервиса, а `/model-info` возвращает метаданные о ML-модели (имя, версию, тип, статус).

13 Проектирование входных данных

Для выполнения прогноза клиент должен передать сведения о хозяйстве. Например:

```
1 {  
2     "farm_id": "FARM-001",  
3     "region": "Krasnodar",  
4     "crop_type": "wheat",  
5     "area_ha": 2500,  
6     "temperature_avg": 24.3,  
7     "precipitation_mm": 320,  
8     "payment_delay_days": 15,  
9     "previous_defaults": 0,  
10    "debt": 1500000  
11 }
```

Листинг 11: Пример входных данных

FastAPI должен проверить эти данные до передачи модели.

14 Pydantic

FastAPI использует Pydantic для описания структуры данных. Добавьте:

```
1 from pydantic import BaseModel, Field
```

Листинг 12: Импорт Pydantic

Создайте:

```
1 class FarmRequest(BaseModel):
2     farm_id: str
3     region: str
4     crop_type: str
5     area_ha: float = Field(gt=0)
6     temperature_avg: float
7     precipitation_mm: float = Field(ge=0)
8     payment_delay_days: int = Field(ge=0)
9     previous_defaults: int = Field(ge=0)
10    debt: float = Field(ge=0)
```

Листинг 13: Модель FarmRequest

Пояснения: `area_ha: float = Field(gt=0)` означает тип данных `float`, значение строго больше 0.

Автоматическая валидация

Если клиент передаст:

```
1 {
2     "area_ha": -200
3 }
```

Листинг 14: Некорректный запрос

FastAPI обнаружит ошибку ещё до запуска нашей функции инференса.

15 Реализация модели

В данной работе вместо модели машинного обучения будем использовать простую функцию, реализующую расчёт риска по некоторым финансовым и отраслевым показателям. Добавьте:

```
1 def calculate_risk(data: FarmRequest) -> float:
2     score = 0.1
3     if data.payment_delay_days > 30:
4         score += 0.3
5     if data.previous_defaults > 0:
6         score += 0.3
7     if data.debt > 5_000_000:
8         score += 0.2
9     if data.precipitation_mm < 100:
10        score += 0.1
11    return min(score, 1.0)
```

Листинг 15: Функция расчёта риска

16 Постпроцессинг

Создайте:

```
1 def risk_level(score: float) -> str:
2     if score < 0.3:
3         return "low"
4     if score < 0.7:
5         return "medium"
6     return "high"
```

Листинг 16: Функция risk_level

Добавьте рекомендации:

```
1 def recommendation(level: str) -> str:
2     if level == "low":
3         return "Standard review"
4     if level == "medium":
5         return "Additional check required"
6     return "High risk. Manual review required"
```

Листинг 17: Функция recommendation

17 Задание 2. Реализуйте POST /predict

Добавьте:

```
1 import uuid
```

Листинг 18: Импорт uuid

Затем:

```
1 @app.post("/predict")
2 def predict(request: FarmRequest):
3     score = calculate_risk(request)
4     level = risk_level(score)
5
6     result = {
7         "request_id": str(uuid.uuid4()),
8         "farm_id": request.farm_id,
9         "risk_score": score,
10        "risk_level": level,
11        "recommendation": recommendation(level),
12        "model_version": "1.0"
13    }
14    return result
```

Листинг 19: Endpoint /predict

18 Проверка через Swagger

Откройте /docs. Выберите POST /predict. Введите:

```
1 {
2     "farm_id": "FARM-001",
3     "region": "Krasnodar",
4     "crop_type": "wheat",
5     "area_ha": 2500,
6     "temperature_avg": 24.3,
7     "precipitation_mm": 320,
8     "payment_delay_days": 45,
9     "previous_defaults": 1,
10    "debt": 6500000
11 }
```

Листинг 20: Тестовый запрос

Получите результат. Например:

```
1 {
2     "request_id": "7c8343c0-c912-4ab0-a56a-a4881894c7af",
3     "farm_id": "FARM-001",
4     "risk_score": 0.9,
5     "risk_level": "high",
6     "recommendation": "High risk. Manual review required",
7     "model_version": "1.0"
8 }
```

Листинг 21: Пример ответа

19 Модель ответа (Response Model)

До настоящего момента мы описывали структуру входа, но не структуру ответа. Создайте:

```
1 class PredictionResponse(BaseModel):
2     request_id: str
3     farm_id: str
4     risk_score: float
5     risk_level: str
6     recommendation: str
7     model_version: str
```

Листинг 22: Модель PredictionResponse

Измените endpoint:

```
1 @app.post(
2     "/predict",
3     response_model=PredictionResponse
4 )
5 def predict(request: FarmRequest):
6     ...
```

Листинг 23: Endpoint /predict с response_model

Внешняя система должна получать ожидаемую структуру данных независимо от внутреннего устройства модели. Модель ответа позволяет контролировать формат ответа, автоматически документировать API, проверять данные ответа, скрывать ненужные внутренние поля и делать контракт между сервером и клиентом явным.

20 Задание 3. Организовать хранение результатов

Создадим временное хранилище в оперативной памяти (в виде словаря):

```
1 predictions = {}
```

Листинг 24: Хранилище прогнозов

Измените `/predict`:

```
1 @app.post(  
2     "/predict",  
3     response_model=PredictionResponse  
4 )  
5 def predict(request: FarmRequest):  
6     score = calculate_risk(request)  
7     level = risk_level(score)  
8     request_id = str(uuid.uuid4())  
9  
10    result = {  
11        "request_id": request_id,  
12        "farm_id": request.farm_id,  
13        "risk_score": score,  
14        "risk_level": level,  
15        "recommendation": recommendation(level),  
16        "model_version": "1.0"  
17    }  
18    predictions[request_id] = result  
19    return result
```

Листинг 25: `/predict` с сохранением результата

21 Path Parameters

Предположим, необходимо получить конкретный прогноз по идентификатору. Запрос: `GET /predictions/abc-123`. Здесь `abc-123` является `path parameter`. FastAPI позволяет записать:

```
1 @app.get("/predictions/{request_id}")  
2 def get_prediction(request_id: str):  
3     ...
```

Листинг 26: Path parameter

Значение из URL попадёт в переменную `request_id`.

22 Задание 4. Получение прогноза

Импортируйте:

```
1 from fastapi import HTTPException
```

Листинг 27: Импорт `HTTPException`

Реализуйте:

```
1 @app.get(  
2     "/predictions/{request_id}",  
3     response_model=PredictionResponse  
4 )  
5 def get_prediction(request_id: str):  
6     if request_id not in predictions:  
7         raise HTTPException(  
8             status_code=404,  
9             detail="Prediction not found"  
10        )  
11     return predictions[request_id]
```

Листинг 28: Endpoint /predictions/{request_id}

HTTP коды ошибок

Обратите внимание на 422. Попробуйте передать:

```
1 {  
2     "farm_id": "FARM-1",  
3     "region": "Krasnodar",  
4     "crop_type": "wheat",  
5     "area_ha": -100,  
6     "temperature_avg": 23,  
7     "precipitation_mm": 200,  
8     "payment_delay_days": 10,  
9     "previous_defaults": 0,  
10    "debt": 10000  
11 }
```

Листинг 29: Некорректные данные

FastAPI должен вернуть ошибку валидации. Обратите внимание на HTTP-код. Разберите структуру ответа.

Ответьте: почему такой запрос не должен доходить до ML-модели?

Ответ: Запрос с `area_ha = -100` не должен доходить до модели, потому что отрицательная площадь физически невозможна. Pydantic отсекает такие данные на этапе валидации (код 422), защищая модель от некорректного входа.

23 Query Parameters

Теперь реализуем получение списка прогнозов. Запрос: `GET /predictions?limit=10`. Здесь `limit=10` является query parameter. Пример:

```
1 @app.get("/predictions")  
2 def get_predictions(limit: int = 10):  
3     values = list(predictions.values())  
4     return values[:limit]
```

Листинг 30: Endpoint /predictions

24 Задание 5. Фильтрация

Добавьте возможность: GET /predictions?risk_level=high. Например:

```
1 @app.get("/predictions")
2 def get_predictions(
3     limit: int = 10,
4     risk_level: str | None = None
5 ):
6     values = list(predictions.values())
7
8     if risk_level is not None:
9         values = [item for item in values if item["risk_level"] ==
10                    risk_level]
11
12     return values[:limit]
```

Листинг 31: /predictions с фильтрацией

Протестируйте: /predictions, /predictions?limit=2, /predictions?risk_level=high, /predictions?risk_level=medium&limit=5.

25 Задание 6. Валидация Query Parameters

Необходимо исключить запрос: GET /predictions?limit=-100. Импортируйте:

```
1 from fastapi import Query
```

Листинг 32: Импорт Query

Измените параметр:

```
1 limit: int = Query(
2     default=10,
3     ge=1,
4     le=100
5 )
```

Листинг 33: Валидация limit

Теперь допустимый диапазон: $1 \leq \text{limit} \leq 100$.

26 Задание 7. Проверка бизнес ограничений

Pydantic проверяет формат данных, но некоторые ошибки невозможно определить только по типу значения, иногда необходима дополнительная проверка некоторых ограничений. Создайте:

```
1 ALLOWED_REGIONS = {
2     "Krasnodar",
3     "Rostov",
4     "Stavropol"
5 }
```

Листинг 34: Множество допустимых регионов

В `/predict` добавьте:

```
1 if request.region not in ALLOWED_REGIONS:
2     raise HTTPException(
3         status_code=400,
4         detail="Unknown region"
5     )
```

Листинг 35: Бизнес-валидация региона

27 Задание 8. Возвращаемые статусы

Добавьте код ответа для `POST /predict`. Например:

```
1 from fastapi import status
2
3 @app.post(
4     "/predict",
5     response_model=PredictionResponse,
6     status_code=status.HTTP_201_CREATED
7 )
```

Листинг 36: Явное указание кода ответа

Вопрос: какой статус корректнее для инференса — «200 OK» или «201 Created»? Архитектор API должен выбрать семантику и использовать её последовательно.

Ответ: Если `/predict` создаёт сохраняемый ресурс (запись прогноза с `request_id`), семантически корректнее 201 Created. В нашей реализации результат сохраняется в словаре `predictions`, поэтому выбран код 201.

28 OpenAPI как контракт

Откройте Swagger UI, теперь в документации автоматически отображаются `FarmRequest`, `PredictionResponse`, типы данных, обязательные поля, ограничения, HTTP endpoints и HTTP methods.

29 Задание 9. Добавьте описание endpoints

FastAPI позволяет сделать документацию более понятной. Например:

```
1 @app.get(
2     "/health",
3     summary="Health check",
4     description="Returns current API status"
5 )
6 def health():
7     ...
```

Листинг 37: Описание `/health`

Для `/predict`:

```

1 @app.post(
2     "/predict",
3     response_model=PredictionResponse,
4     summary="Farm risk assessment",
5     description="Runs inference of the agro scoring model"
6 )
7 def predict(request: FarmRequest):
8     ...

```

Листинг 38: Описание /predict

Проверьте изменение Swagger UI.

30 Задание 10. Тестирование API

Последовательно выполните запросы и заполните таблицу тест-кейсов.

Таблица 4: Таблица тест-кейсов

№	Запрос	Ожидаемый HTTP-код	Полученный код	Результат
1	GET /health	200	200	успех
2	GET /model-info	200	200	успех
3	корректный POST /predict	201	201	успех
4	area_ha = -100	422	422	успех
5	неизвестный регион	400	400	успех
6	существующий request_id	200	200	успех
7	неизвестный request_id	404	404	успех
8	limit = 2	200	200	успех
9	risk_level = high	200	200	успех
10	limit = -5	422	422	успех

31 Задание 11. Проанализируйте API как контракт

Для каждого endpoint заполните таблицу.

Таблица 5: Анализ API-контракта

Endpoint	Method	Input	Output	Success	Возможные ошибки
/health	GET	—	status	200	500
/model-info	GET	—	model info	200	500
/predict	POST	FarmRequest	PredictionResponse	201	400, 422, 500
/predictions/{id}	GET	path id	PredictionResponse	200	404
/predictions	GET	query params	list	200	422

32 Задание 12. Разделение приложения на модули

Разделите приложение на несколько файлов:

```
1 agro_api/  
2     main.py  
3     schemas.py  
4     model.py  
5     services.py  
6     storage.py
```

Листинг 39: Структура проекта после рефакторинга

schemas.py содержит: `FarmRequest`, `PredictionResponse`.

model.py содержит: `calculate_risk()`.

services.py содержит: `risk_level()`, `recommendation()`, `is_valid_region()`, `build_prediction()`.

storage.py содержит: `predictions`, `save_prediction()`, `get_prediction()`, `list_predictions()`.

main.py содержит: инициализацию FastAPI и маршрутизацию endpoints.

Отчёт должен содержать:

1. Цель работы.
2. Краткое описание архитектуры API.
3. Исходный код приложения: https://github.com/dashakomarova2002-commits/HW_1_SISS
4. Реализованные endpoints: GET /health, GET /model-info, POST /predict, GET /predictions/{region_id}, GET /predictions.
5. Скриншот Swagger UI.
6. Скриншот успешного /predict.
7. Скриншот ошибки Pydantic.
8. Скриншот 404 Not Found.
9. Таблицу тест-кейсов.
10. Таблицу API-контракта.
11. В качестве вывода опишите разработанную архитектуру, желательно нарисовать компонентную схему архитектуры. Также сделайте краткий вывод о роли REST API в AI-системе.

33 Критерии оценки

Таблица 6: Критерии оценки

Критерий	Баллы
Приложение запускается на FastAPI	5
Реализован /health	5
Реализован /model-info	5
Реализован /predict	15
Используется Pydantic	10
Реализована структурная валидация	10
Реализована бизнес-валидация	10
Реализован поиск по request_id	10
Используются path/query parameters	10
Обрабатывается HTTP 404	5
Корректно используются HTTP-коды	5
Проведено тестирование	5
Проанализирован OpenAPI-контракт	5
Итого	100

34 Скриншоты работы приложения

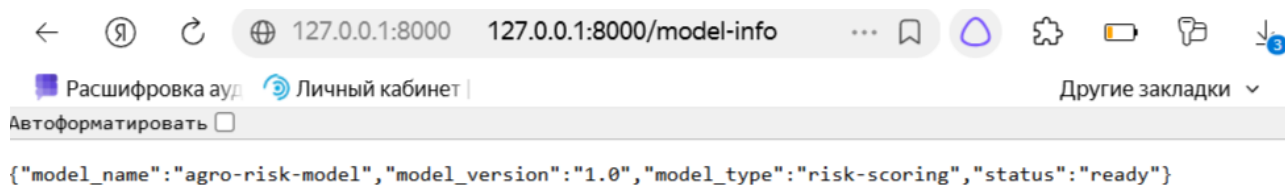


Рис. 1: Swagger UI со всеми endpoints

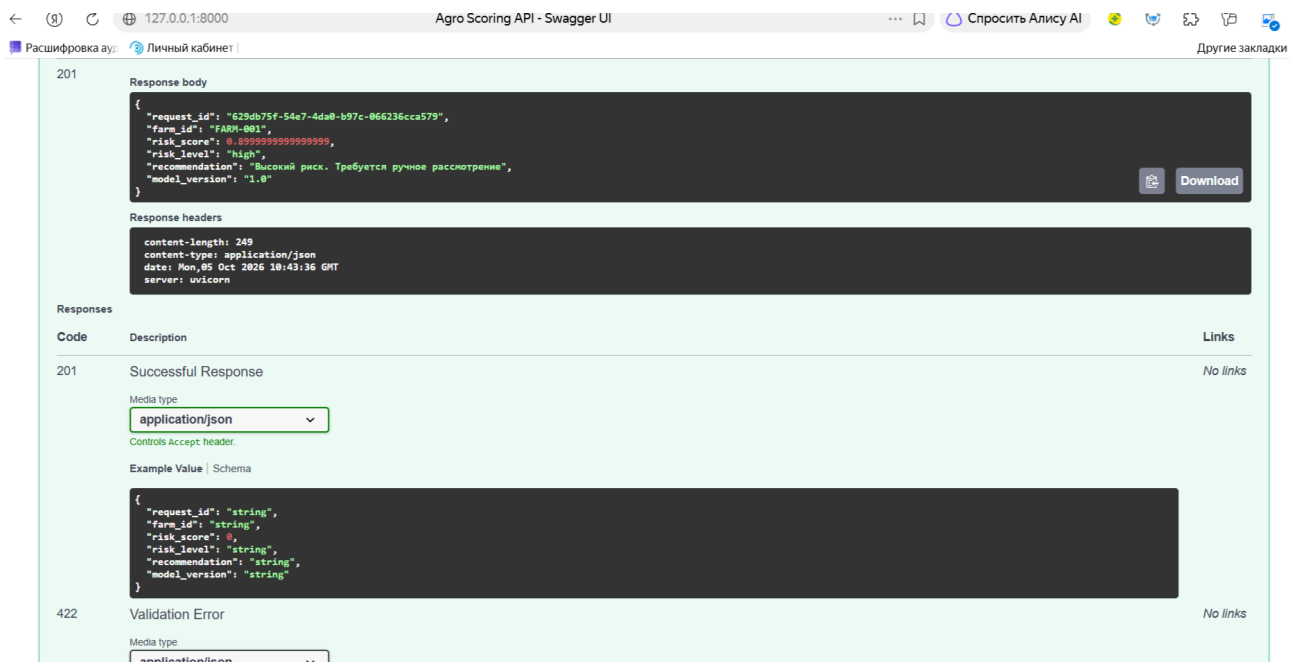


Рис. 2: Успешный POST /predict (код 201)

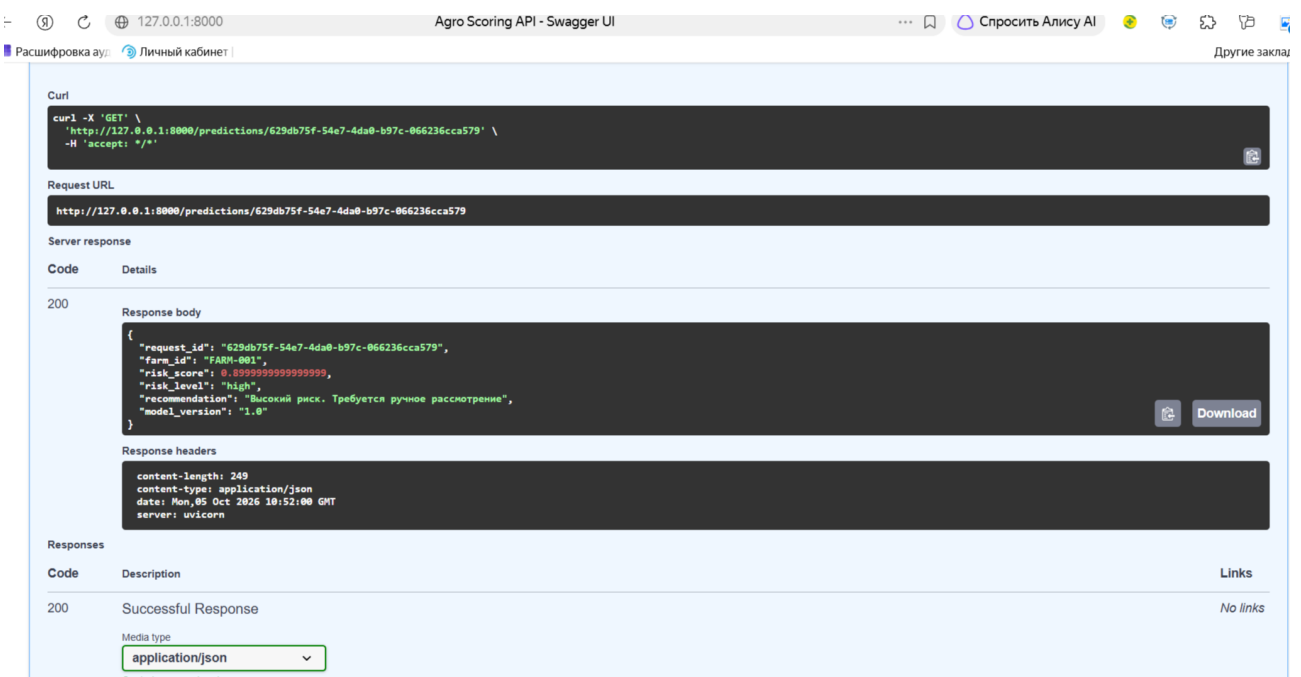


Рис. 3: Получение прогноза по request_id

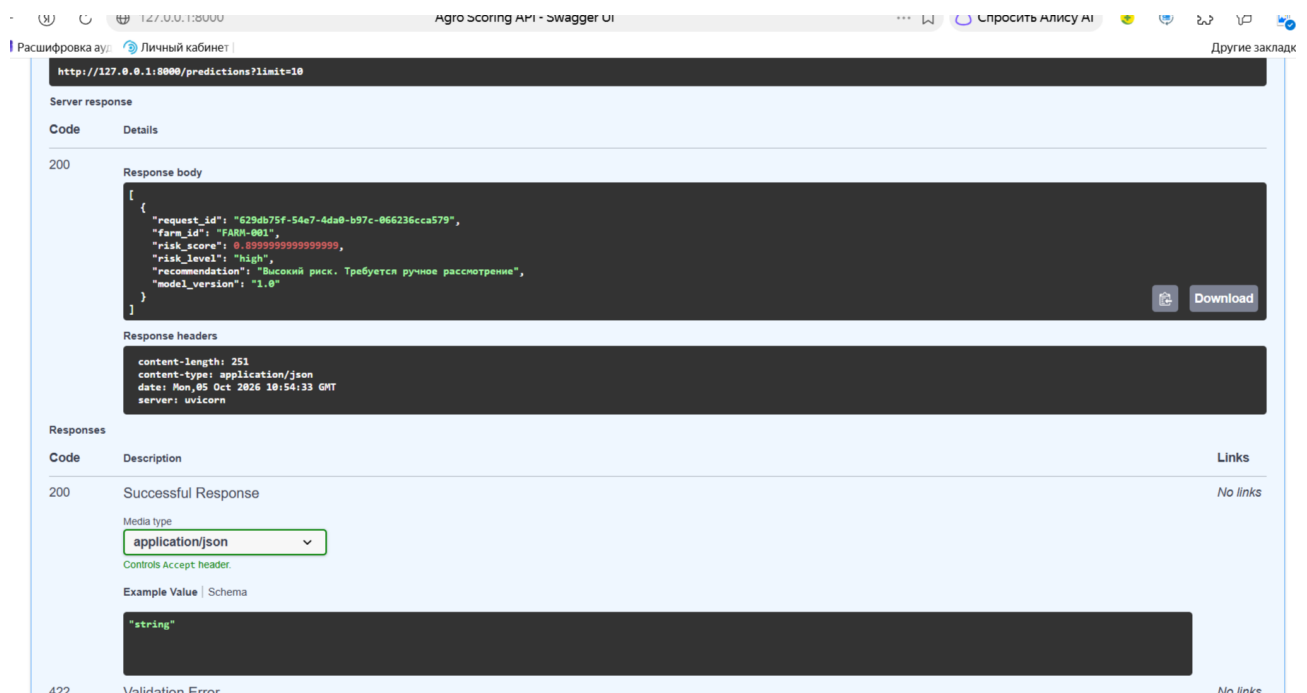


Рис. 4: Список прогнозов с фильтрацией

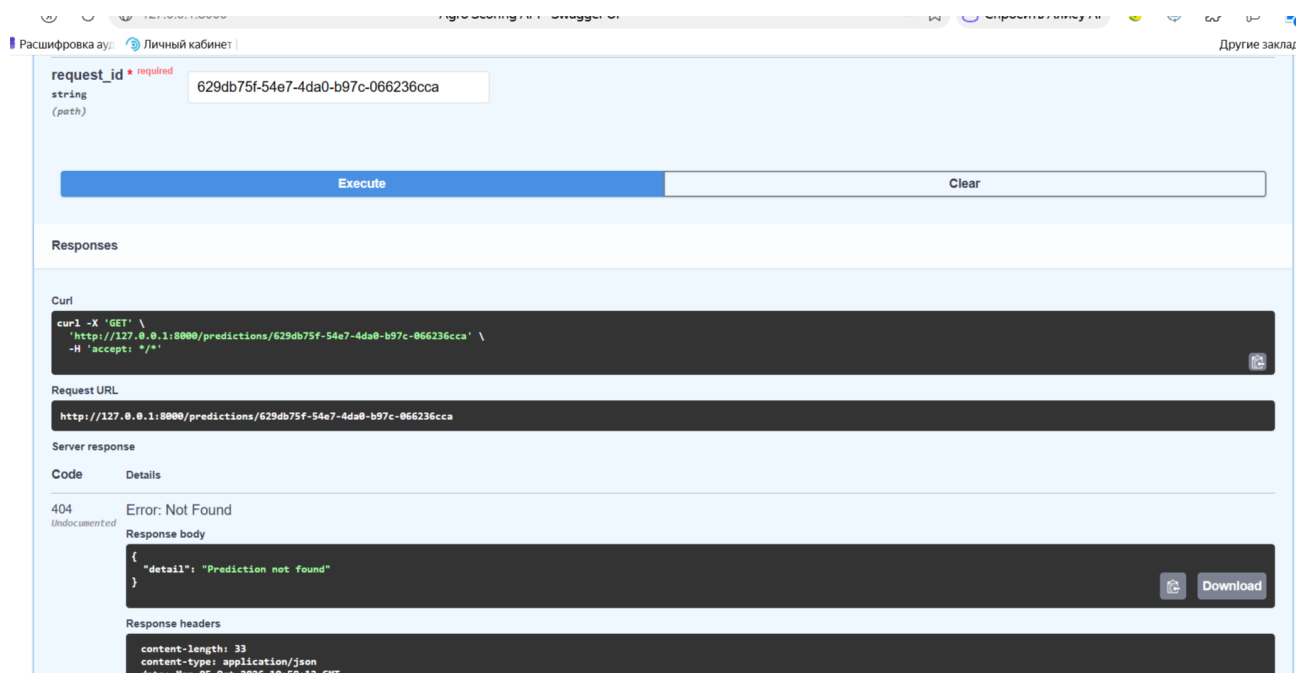


Рис. 5: Ошибка 404 Not Found



Рис. 6: Ошибка валидации Pydantic (422)



Рис. 7: Ошибка бизнес-валидации региона (400)

Приложение А: Полный код приложения

```
1 from pydantic import BaseModel, Field
2
3
4 class FarmRequest(BaseModel):
5     farm_id: str
6     region: str
7     crop_type: str
8     area_ha: float = Field(gt=0)
9     temperature_avg: float
10    precipitation_mm: float = Field(ge=0)
11    payment_delay_days: int = Field(ge=0)
12    previous_defaults: int = Field(ge=0)
```

```
13     debt: float = Field(ge=0)
14
15
16 class PredictionResponse(BaseModel):
17     request_id: str
18     farm_id: str
19     risk_score: float
20     risk_level: str
21     recommendation: str
22     model_version: str
```

Листинг 40: schemas.py

```
1 from schemas import FarmRequest
2
3
4 def calculate_risk(data: FarmRequest) -> float:
5     score = 0.1
6     if data.payment_delay_days > 30:
7         score += 0.3
8     if data.previous_defaults > 0:
9         score += 0.3
10    if data.debt > 5_000_000:
11        score += 0.2
12    if data.precipitation_mm < 100:
13        score += 0.1
14    return min(score, 1.0)
```

Листинг 41: model.py

```
1 from schemas import FarmRequest
2 from model import calculate_risk
3
4 ALLOWED_REGIONS = {"Krasnodar", "Rostov", "Stavropol"}
5
6
7 def risk_level(score: float) -> str:
8     if score < 0.3:
9         return "low"
10    if score < 0.7:
11        return "medium"
12    return "high"
13
14
15 def recommendation(level: str) -> str:
16     if level == "low":
17         return "Standard review"
18     if level == "medium":
19         return "Additional check required"
20     return "High risk. Manual review required"
21
22
23 def is_valid_region(region: str) -> bool:
24     return region in ALLOWED_REGIONS
```

```
25
26
27 def build_prediction(request: FarmRequest, request_id: str) -> dict:
28     score = calculate_risk(request)
29     level = risk_level(score)
30     return {
31         "request_id": request_id,
32         "farm_id": request.farm_id,
33         "risk_score": score,
34         "risk_level": level,
35         "recommendation": recommendation(level),
36         "model_version": "1.0"
37     }
```

Листинг 42: services.py

```
1 predictions: dict = {}
2
3
4 def save_prediction(request_id: str, result: dict) -> None:
5     predictions[request_id] = result
6
7
8 def get_prediction(request_id: str) -> dict | None:
9     return predictions.get(request_id)
10
11
12 def list_predictions(limit: int = 10, risk_level: str | None = None) ->
13     list:
14     values = list(predictions.values())
15     if risk_level is not None:
16         values = [item for item in values if item["risk_level"] ==
17                     risk_level]
18     return values[:limit]
```

Листинг 43: storage.py

```
1 import uuid
2
3 from fastapi import FastAPI, HTTPException, Query, status
4
5 from schemas import FarmRequest, PredictionResponse
6 from services import build_prediction, is_valid_region
7 from storage import save_prediction, get_prediction, list_predictions
8
9 app = FastAPI(
10     title="Agro Scoring API",
11     description="API for agricultural risk scoring",
12     version="1.0.0"
13 )
14
15
16 @app.get("/health", summary="Health check")
17 def health():
```

```
18     return {"status": "ok"}
19
20
21 @app.get("/model-info")
22 def model_info():
23     return {
24         "model_name": "agro-risk-model",
25         "model_version": "1.0",
26         "model_type": "risk-scoring",
27         "status": "ready"
28     }
29
30
31 @app.post(
32     "/predict",
33     response_model=PredictionResponse,
34     status_code=status.HTTP_201_CREATED,
35     summary="Farm risk assessment"
36 )
37 def predict(request: FarmRequest):
38     if not is_valid_region(request.region):
39         raise HTTPException(status_code=400, detail="Unknown region")
40
41     request_id = str(uuid.uuid4())
42     result = build_prediction(request, request_id)
43     save_prediction(request_id, result)
44     return result
45
46
47 @app.get(
48     "/predictions/{request_id}",
49     response_model=PredictionResponse
50 )
51 def get_prediction_endpoint(request_id: str):
52     result = get_prediction(request_id)
53     if result is None:
54         raise HTTPException(status_code=404, detail="Prediction not
55                             found")
56     return result
57
58 @app.get("/predictions")
59 def get_predictions(
60     limit: int = Query(default=10, ge=1, le=100),
61     risk_level: str | None = None
62 ):
63     return list_predictions(limit=limit, risk_level=risk_level)
```

Листинг 44: main.py

Вывод

В ходе работы спроектирован и реализован AI-сервис агроскоринга на FastAPI. Архитектура включает: API-слой (FastAPI), слой валидации (Pydantic-схемы `FarmRequest` и `PredictionResponse`), контур инференса (функция `calculate_risk` и постпроцессинг `risk_level` и `recommendation`) и хранилище результатов (словарь `predictions` в оперативной памяти).

REST API в AI-системе выполняет три ключевые роли: точка входа для всех внешних потребителей (CRM, портал, мобильное приложение), изолятор деталей реализации модели и контракт, фиксирующий форматы данных, коды ответов и правила обработки ошибок. Благодаря этому модель можно заменить или переобучить без изменения кода клиентов.

Компонентная схема:

